Slicing Quantum Programs for Comprehension: What a Quantum Slice Should Look Like

Anonymous Author(s)

I. Introduction

A developer who sees a qubit end in the wrong state asks which lines of the program determine it. For classical programs, a backward slice answers that question [1], and programmers use slices when they read and debug code [2], [3]. Quantum programs need such help more. Their intermediate states cannot be inspected without disturbing them, and a gate on one qubit can change another, as in phase kickback, where a controlled gate changes the qubit that controls it. Quantum developers debug with print statements, circuit drawings, simulators and manual reading of the code, and report that these methods do not scale [4]. Bug detectors such as LintQ and QChecker find known patterns by static analysis, which reads a program without running it, but many faults are wrong outputs that no pattern describes [5], [6]. A slice that shows where such a fault can be is the natural aid.

Slicing has been proposed for quantum programs [7], [8], [9], but no proposed dependence rule has been checked against what a quantum program computes. For comprehension this matters more than for any other use of slicing, because a slice that hides code able to change the qubit misleads its reader, who will not look where the fault is. Such slices also look right. Fig. 1 shows a textbook program whose classical slice reproduces the faulty program's state and leaves out the faulty line. Over the 784 circuit–qubit pairs of our study, reading a controlled gate the classical way, as an assignment that leaves its control unchanged, gives a wrong slice for 26.0% of the pairs, and 43% of those wrong slices reproduce the outcome histogram of the whole program, so a developer who checks a slice by its outcomes would accept them.

This paper asks what a quantum slice must keep, and which dependence rule produces it. We define a slice of a set of qubits by what it must preserve, their joint state for every input, and an operation as a dependence of those qubits if a change made right after it can change that state. One rule meets both, the *write-all rule*, under which a gate writes every qubit it touches, its controls included. On qubits its slice is the backward light cone, whose soundness compilers already rely on [10], [11]. What has been missing is the case for it as a slice, that it keeps every dependence, that nothing else proposed for quantum programs adds a dependence it misses, and what each smaller or different rule loses. We give the case in proofs and measure the losses in a pre-registered study of every rule a developer or tool builder would otherwise use, on 158 benchmark circuits with an exact ground truth, 119 generated circuits and 5,688 seeded-fault units, with real faults from Bugs4Q [12].

This paper makes the following contributions.

1) **Semantics.** A definition of what a slice of a set of qubits must preserve and of when an operation is a dependence of those qubits, which separates sound slices from slices that keep every dependence.
2) **The rule and its guarantees.** The slice of the quantum dependence graph (QDG), which the write-all rule defines and classical bits extend from a measurement to the operations it conditions, is sound for every input, with the induction mechanized in Isabelle/HOL, keeps every dependence and distributes over criteria. For each of the 21 two-qubit gates of Qiskit's standard library, either qubit can change the other, so no gate can safely write fewer qubits.
3) **Three kinds of dependence.** Classical data, classical control and quantum data dependence are all a slice needs. What looks like quantum control dependence is quantum data dependence, and entanglement is neither necessary nor sufficient for dependence, so entanglement edges can only add code that cannot matter.
4) **What the alternatives cost.** The classical program dependence graph misses dependences almost only through its reading of the control, 99.7% of them. The QPDG of Yu and Zhao [7] is sound, but its slices hold 2.7 times as much code that cannot change the criterion as the QDG's, whose slices come within 4.8 points of the smallest slice that keeps every dependence. Pruning a slice by commutation or by simulating one input makes it smaller but drops dependences, and a state-based slice stays correct for random inputs on only 55.4% of the pairs. Slices narrow where a developer looks but do not improve mutation-based fault ranking or a language model's localization of faults.

The paper does not build a slicer for a particular language. Its definitions are stated on the circuit a program builds, Section IV-E states how a slice reaches the program's source lines, and computing slices at scale on the source developers write is a separate question. Section II reviews related work and Section III works through the example of Fig. 1. Section IV defines the semantics and the rules, and Sections V and VI present the study. The paper ends with the implications for comprehension tools, the threats to validity and the conclusion. The data, scripts, protocol and proofs are available in an anonymous replication package.[TODO: anonymous package link]

II. Related Work

This section places our work among five lines of research. For each line it states what others have done, how our work relates to it and what we reuse. It closes with the gaps that remain open and with Table I, which compares the dependence models that have been proposed or used for quantum programs.

A. Program slicing and its semantics

Weiser introduced program slicing [1] and showed that programmers use slices when they debug [2]. Slicing extracts a computation that is scattered through a program from the statements around it, which makes it a comprehension aid as much as a debugging one [3]. Dependence graphs reduced slicing to graph reachability [13], [14], [15], Podgurski and Clarke gave dependence a semantic definition [16], and dynamic slicing restricts a slice to one execution [17], [18]. A backward slice answers which statements can affect a value at a point, and a forward slice, its dual, answers which statements a given statement can affect [19], [15]. Slicers need not build a graph, and srcSlice, for example, computes dependences from a slice profile of every variable [20]. Surveys catalog the many kinds of slices that followed [19], [21], [22]. Probabilistic slicing preserves the distribution of the criterion variables [23], [24], [25], and Gallagher names quantum programs as a direction for semantically defined slicing [26].

Relation and reuse. We keep the classical notions, a criterion, a backward and a forward slice, and dependence as the possibility of change, and define them for the reduced state of a set of qubits, a criterion stronger than a distribution in one basis, which misses phase.

B. Dependence models for quantum programs

Yu and Zhao's quantum program dependence graph (QPDG) [7] extends the classical PDG [14] with entanglement, no-cloning, temporal and measurement dependences and proposes slicing on it. It is defined in prose with examples and has not been implemented or evaluated. Its own example shows why validation matters. In the graph Yu and Zhao draw for their first listing, the slice of a measured qubit omits the two gates that fix its outcome, so the slice returns 1 with probability 1/2 where the program returns 1 with certainty. Kaul et al. build the quantum code property graph (QCPG), a code property graph for Qiskit and OpenQASM programs, and query it for bugs and smells, among them superfluous operations, gates that do not affect a measured qubit [8]. Its data-flow edges run from the control of a CNOT to its target only, and the authors state that the control is not changed, so the query can report gates whose effect reaches a measured qubit back through a control. Kaul et al. demonstrate the graph on example programs and note the lack of large datasets. Behler et al. derive the order of qubit operations from OpenQASM 3 source and use it to detect patterns and suggest refactorings, and their order table marks the control of a CNOT apart from its target [27], [28]. In their roadmap of quantum software engineering, Murillo et al. describe the QPDG as an aid to understanding programs [9], and in their roadmap of testing and debugging, Ramalho et al. find dependence and data-flow analysis of circuits almost unexplored, naming only Kaul et al. [29].

Relation and reuse. We take the QPDG's question, which operations a qubit depends on, and answer it against a semantic definition. Section IV-D describes the baseline we reimplement from its prose and the classical reading of the control that the QCPG states, and Section VI measures both.

C. Entanglement analysis

Static analyses approximate which qubits may be entangled, with type systems [30], [31], [32], abstract interpretation [33], [34], the stabilizer formalism [35] and data-flow rules [36]. ScaffCC assumes that qubits that interact may be entangled [37].

Relation. These analyses answer a different question, whether two qubits may be correlated, and they are not slicers. Section IV shows that entanglement neither implies nor is implied by dependence, so their results cannot decide what a slice keeps.

D. Light cones and compilers

The *backward light cone* of a set of qubits at the end of a circuit is the set of gates that can affect their final state. Compilers build gate-dependence graphs on this principle to schedule and map circuits [10], [38], and QAOA analyses use it to bound the gates that can reach an observable [39]. Quantum separation logics reason locally about the qubits a program touches [40], [41]. Qiskit implements the light cone as the `LightCone` transpiler pass [11]. Given the qubits to be measured, or a Pauli observable on them, the pass walks the circuit backward from the end, keeps the gates in the light cone and also removes every gate that commutes with the observable, since such a gate cannot change its expectation value. It returns a smaller circuit for estimating that observable, decides commutation for the circuit's actual angles and rejects circuits with mid-circuit measurements. Heim et al. likewise argue that quantum dependences should account for commutation [42]. Compilers also call the time windows of a circuit its slices [43], a sense different from ours.

Relation and reuse. On qubits, the backward light cone is the relation of our write-all rule, and that it determines the final state of its qubits is known. We claim neither the relation nor its soundness as new. What these works do not provide is a definition of what a slice must preserve, a proof that the light cone keeps every dependence, and a measurement of what smaller or different rules lose, which Sections IV and VI give. Removing commuting gates goes further than the light cone. A pruned slice is correct for the observable at the given angles but no longer contains every operation that could change the qubit, so it can omit the place where a fault would be. We reuse Qiskit's `LightCone` pass as the commutation-pruning baseline and follow the light cone along classical bits, which the pass does not do.

E. Debugging tools and bug detectors

Metwalli and Van Meter adapt slicing to circuits by splitting a circuit at barriers the developer places and removing the qubits a part does not use [44], [45], and Qiskit's circuit-slicing utilities cut a circuit into consecutive parts by barriers, depth or gate type [46]. Both cut a circuit into pieces to inspect and follow no dependence. Ramalho et al. list slicing as a way around no-cloning and note that it grows hard to manage as circuits grow [29]. Zappin et al. surveyed and interviewed 26 quantum developers, who debug with print statements, circuit drawings, simulators and manual inspection of the code and report that these methods do not scale [4]. Testing faces the limit that phase faults escape computational-basis measurement [47]. Bug detectors find known patterns, and Chen et al. catalog quantum-specific code smells [48]. LintQ runs queries over Qiskit programs and orders two operations when they share a qubit [5], QChecker matches patterns in the abstract syntax tree [6], and Q-PAC also detects known patterns [49]. Bug patterns [50], Bugs4Q [12], mutation tools [51], [52], [53] and mutation-based fault localization [54] supply faults and rankings to this field.

Relation and reuse. Slices complement these tools. Manual slicing has no criterion, so the developer still narrows the fault by hand, and a slice by the write-all rule generalizes the removal of unused qubits, since it drops every qubit and operation that cannot reach the criterion. LintQ's ordering relation is the relation of the write-all rule, which confirms the rule in practice, while LintQ uses it for patterns and makes no claim about slices. Of the 42 Bugs4Q bugs on which QChecker was evaluated, 22 are wrong outputs that no pattern detects [6], and a slice shows where such a fault can be. We reuse Bugs4Q, published mutation operators and mutation-based fault localization.

F. Open gaps and the data used

Table I compares the models of dependence these lines of work use, by what a CNOT writes, whether measured bits are followed, which extra edges are added, and what is defined and guaranteed about a slice. Three gaps remain open, and each can be shown from the work above. First, no dependence rule proposed for quantum slicing has been checked against what a quantum program computes, and the one example worked out in detail, the QPDG's, gives a wrong slice. Second, the analyses that represent quantum programs for static analysis, the QCPG and the order tables of Behler et al., read the control of a gate as unchanged, and no work has measured what that reading does to a slice. Third, the light cone is known to be sound, but no work has asked whether it keeps every dependence, how far it is from the smallest slice that does, and what smaller rules lose in correctness and in locating faults.

The data differ as much as the models. The dependence graphs were shown on examples and the detectors evaluated on Bugs4Q and on LintQ's GitHub corpus, while no rule for quantum slicing was evaluated against a ground truth. We use Bugs4Q for real faults and add what a study of rules needs, circuits with a simulated ground truth and seeded faults, which Section V lists.

CPDG QDG **Bernstein–Vazirani** (Qiskit textbook), slices of q_0

```
16 ■ ■ n = 3
17 □ ■ s = '011'
21 ■ ■ bv_circuit = QuantumCircuit(n+1, n)
24 □ ■ bv_circuit.h(n)
25 □ ■ bv_circuit.x(n) ← fault, textbook: z(n)
28 ■ ■ for i in range(n):
29 ■ ■     bv_circuit.h(i)
32 □ □ bv_circuit.barrier()
35 □ ■ s = s[::-1]
36 □ ■ for q in range(n):
37 □ ■     if s[q] == '0':
38 □ □         bv_circuit.i(q)
39 □ ■     else:
40 □ ■         bv_circuit.cx(q, n)
43 □ □ bv_circuit.barrier()
46 ■ ■ for i in range(n):
47 ■ ■     bv_circuit.h(i)
50 □ □ for i in range(n):
51 □ □     bv_circuit.measure(i, i)
```

outcomes $q_2q_1q_0$: buggy 000 (1.00); intended 011 (1.00)

Fig. 1. The Bernstein–Vazirani program of the Qiskit textbook [55], with comments and blank lines omitted and the textbook's line numbers kept. Line 25 carries a seeded fault, `x(n)` where the textbook applies `z(n)`. The two columns mark the lines in the backward slice of q_0 under the classical PDG and under the QDG. The classical slice misses the orange lines, which can change q_0. Each slice holds the lines of its operations and the host lines that compute their qubits, as Section IV-E describes. Slices, outcomes and dependences are computed by the study's code.

III. A Classical Slice That Looks Right

This section shows, on a textbook program, how a slice computed the classical way can reproduce the output a developer sees and still hide the fault, and why a check of measured outcomes cannot tell. It starts with the quantum concepts the rest of the paper uses.

Background: A qubit's state is a superposition $\alpha|0\rangle + \beta|1\rangle$, and measuring it yields 0 or 1 with probabilities $|\alpha|^2$ and $|\beta|^2$. The two states $|+\rangle$ and $|-\rangle$ give both outcomes equally often and differ only in the sign of β, their phase, which no measurement in the computational basis can see. Gates are reversible operations on qubits, such as the Hadamard gate H, which turns $|0\rangle$ into $|+\rangle$ and $|+\rangle$ back into $|0\rangle$. A controlled gate such as the controlled-NOT (CNOT) applies an operation to its target depending on its control, but it can also change the control. When the target is in $|-\rangle$, the CNOT leaves the target as it is and flips the phase of its control instead, an effect called phase kickback. Two qubits are entangled when their joint state cannot be written as a state of each qubit alone. A measurement writes a classical bit, which later gates may read as a condition. Qiskit programs build circuits in Python, with

TABLE I
DEPENDENCE MODELS FOR QUANTUM PROGRAMS, BY WHAT THEY READ A GATE TO DO AND WHAT THEY GUARANTEE

Approach	CNOT writes	Measured bits	Extra edges	Slice semantics	Guarantee	Evaluated on
QPDG, Yu and Zhao [7]	?	●	entanglement, closed	○	none	5 listings, not implemented
QCPG, Kaul et al. [8]	t	●	none	○	none	3 listings, 1 Qiskit/OpenQASM circuit
Behler et al. [27], [28]	t	◐	none	○	none	16 examples of OpenQASM 3
LintQ [5]	c, t	●	none	○	none	7,568 GitHub programs, 216 confirmed bugs
TriQ, staq [10], [38]	c, t	–	none	○	sound, known	12 and 24 benchmark circuits
Qiskit `LightCone` [11]	c, t	○	none, removes commuting gates	◐	observable, fixed angles	–
QDG, this paper	c, t	●	none	●	sound, complete	784 circuit–qubit pairs of 158 circuits, 119 generated and 55 large circuits, 5,688 seeded and 15 Bugs4Q fault units

● has the property, ◐ has it in part, ○ lacks it, ? means the authors do not specify it, and – means it does not apply. *CNOT writes*, the qubits a CNOT with control c and target t is read to change, from which the dependences follow. *Measured bits*, classical control is followed from a measurement to the operations its bit conditions. *Extra edges*, dependences added beyond those of the gates' qubits. *Slice semantics*, what a slice must preserve is defined. *Guarantee*, the property of a slice that the authors state or prove; complete means that it keeps every dependence. *Evaluated on*, the data of the original evaluation.

loops, functions and conditions that compute which gates act on which qubits.

The program: The Bernstein–Vazirani algorithm [56] finds a hidden bit string s with a single query to an oracle. Fig. 1 shows the program of the Qiskit textbook [55], with $s = 011$. It rests on phase kickback. Lines 24 and 25 prepare the auxiliary qubit q_3 in $|-\rangle$, line 29 puts every data qubit in $|+\rangle$, and for every bit of s that is 1, the CNOT of line 40 kicks a phase onto its data qubit, turning $|+\rangle$ into $|-\rangle$. The Hadamard gates of line 47 turn these phases into the bits of s, so the program returns 011. We seeded one fault, a one-token change on line 25 that applies X instead of Z. The auxiliary is then left in $|+\rangle$, on which a CNOT changes nothing, and the program returns 000 instead of 011.

The slices: The developer sees q_0 read 0 where s says 1, and slices backward from q_0 to see which lines determine it. A classical slicer reads `cx(q, n)`, which flips its target when its control is 1, as the assignment $q_n \leftarrow q_n \oplus q$, which reads q and writes only the auxiliary. Its slice of q_0 keeps the Hadamard gates of lines 29 and 47 and the lines that compute their qubits, lines 16, 21, 28 and 46, as the left column of Fig. 1 shows. The slice looks right. Its 2 operations give q_0 the same state as the faulty program, with fidelity 1.00, a similarity of quantum states that is 1 exactly when they are equal, so it gives the developer no reason to look at lines 24, 25 and 40, where the fault is.

These lines can nonetheless change q_0. Restoring Z on line 25 turns q_0 from $|0\rangle$ into $|1\rangle$, two states with fidelity 0.00, because the CNOT acts on its control as well as its target. A slice that lets every gate write all its qubits, the QDG of Section IV, keeps lines 24, 25, 29, 40 and 47, together with the lines that compute them, among them the oracle's loop and condition on lines 35 to 39. Its slice contains the fault. The QPDG of Yu and Zhao keeps the same lines here, since its two extra operations, the Hadamard gate and the CNOT of q_1, are later iterations of lines 29 and 40 that cannot change q_0, and Fig. 2 shows where it costs more.

Why the outcomes cannot tell: Here the fault shows in the measured outcome of q_0. That is not always so. In a real bug from Bugs4Q [12], posted on Stack Overflow, a two-qubit program meant to prepare $(|01\rangle + |10\rangle)/\sqrt{2}$ applies `h(0)`, `h(1)` and `cx(0,1)`, with `h(1)` where `x(1)` was needed, and returns all four bit strings equally often. Each qubit alone reads 1 with probability 0.5, in the buggy program and in the intended one alike, so the outcomes of either qubit cannot reveal the failure, and only their joint outcomes do. The fault does change the state of q_0, which is a uniformly random bit in the intended program and the pure $|+\rangle$ in the buggy one, two states with fidelity 0.50. Because the CNOT writes its control, the QDG slice of q_0 contains the faulty `h(1)`, while the classical slice keeps only `h(0)`.

Both lessons hold beyond these programs, as Section VI shows. Reading a gate as an assignment misses influence that flows into a control qubit, and judging a slice by the outcomes of one qubit misses errors in phase and in correlations. A slice must therefore be judged by the state of the qubits it is about, which is how Section IV defines it.

IV. QUANTUM DEPENDENCE AND THE WRITE-ALL RULE

This section defines what a slice of a set of qubits must preserve and when an operation is a dependence of those qubits. It then states the write-all rule, proves that it gives slices that are sound and contain every dependence, shows that three kinds of dependence are all a slice needs, and places every other rule relative to it. The definitions do not depend on any slicer, and Section VI measures each rule against them.

A. Circuits, criteria and dependence

A Qiskit program does not act on qubits directly. It builds a *circuit*, which a simulator or a quantum computer then runs. The circuit $P = o_0 \dots o_{L-1}$ is the sequence of operations the program appends, in the order it appends them, and position

k holds the operation o_k. The circuit has n qubits, numbered $Q = \{0, \ldots, n-1\}$, which form its quantum memory, and a set C of classical bits, which store the outcomes of measurements.

Each operation touches some qubits and bits, which we call its *locations* $loc(o) \subseteq Q \cup C$. A gate touches the qubits it is applied to, so `h(0)` has the location q_0 and `cx(0, 1)` the locations q_0 and q_1. A measurement touches the measured qubit and the bit that receives its outcome, so `measure(2, 0)` has the locations q_2 and c_0. A reset, which returns a qubit to $|0\rangle$, touches that qubit. A conditioned gate is applied only when a measured bit has a given value, as in `x(0)` inside `if_test((c[0], 1))`, and it touches its qubits and the bits its condition reads, here q_0 and c_0.

The program's own loops and branches run while the circuit is built and decide which operations it contains and how often. Loops that run on the device, inside the circuit, are outside this model, and none of the study's circuits has one.

The state of the qubits is described by a density matrix ρ, which also covers the mixtures that arise when a measurement outcome is not known in advance. Running P on the input ρ, with every bit initially 0, gives $[\![P]\!](\rho)$, the final joint state of qubits and bits, averaged over the outcomes of the measurements. A developer asks about some of the qubits, a set $W \subseteq Q$. Their *reduced state* $\rho_W = \mathrm{Tr}_{\overline{W}}(\rho)$ is what remains of ρ when everything else, $\overline{W} = (Q \setminus W) \cup C$, is traced out, that is, averaged away. It determines every statistic of measuring W in any basis, so two runs that give W the same reduced state cannot be told apart by looking at W. A slice is a set S of positions in P, and it is judged by running the restriction $P|_S$, the circuit that keeps only the operations at the positions in S, in their original order.

Definition 1 (Slicing criterion): A slicing criterion is a nonempty set $W \subseteq Q$ of qubits, whose state is taken at the end of the circuit, before any terminal measurement reads it.

The criterion is a set of qubits, not a single one, because the joint state of entangled qubits is not determined by their separate states, so a developer may need to ask about several qubits together. Yu and Zhao keep the classical criterion of a variable at a statement [7], and we preserve the *joint* state of the criterion instead. Taking that state before readout keeps a slice correct when it is composed with later gates or read in another basis.

Definition 2 (Soundness): A set of positions S *preserves* W at input ρ if $\mathrm{Tr}_{\overline{W}}[\![P|_S]\!](\rho) = \mathrm{Tr}_{\overline{W}}[\![P]\!](\rho)$. It is a *sound* slice for W if it preserves W at every input.

A sound slice reproduces the criterion, but comprehension needs more, the assurance that the slice shows every place where a change, and so a fault, could alter the criterion.

Definition 3 (Dependence): Operation o_k is a *dependence* of W in P if there are an input ρ and a unitary U on the qubits of o_k such that inserting U immediately after o_k changes $\mathrm{Tr}_{\overline{W}}[\![P]\!](\rho)$. For a measurement, negating the bit it writes also counts as such a change. $Dep_W(P)$ is the set of dependences of W, and a set S is *dependence-complete* for W if $Dep_W(P) \subseteq S$.

This is semantic dependence in the sense of Podgurski and Clarke [16], stated for quantum states. The two guarantees differ. A sound slice need not contain every dependence. The identity gate of line 38 in Fig. 1, which acts on q_2, is a dependence of q_2, since a perturbation inserted after it changes the state of q_2, yet deleting it changes nothing. In the other direction, deleting a single unitary operation that is not a dependence never changes ρ_W, since deleting it has the same effect as inserting its inverse after it.

B. The write-all rule

A *dependence rule* R assigns each operation the locations it reads, $r_R(o)$, and the locations it writes, $w_R(o)$. Operation o_j depends on o_i under R if $i < j$ and some location in $w_R(o_i) \cap r_R(o_j)$ is not written by an operation between them. The backward slice $B_R(W)$ is the set of operations from which the last writer of some $q \in W$ is reachable along these dependences. The rules differ in one choice, what an operation writes.

Definition 4 (Write-all rule): Every operation reads and writes all of its locations, $r(o) = w(o) = loc(o)$. In particular a controlled gate writes its controls as well as its targets. The *quantum dependence graph* (QDG) has the operations as nodes and the dependences of this rule as edges, and the QDG slice $B_{\mathrm{QDG}}(W)$ is backward reachability from W.

The QDG specifies the slice. Computing the slice needs no graph, only one backward pass over the circuit. Start at the end of the circuit with the needed set $N = W$, and whenever an operation acts on a location in N, keep the operation and add all its locations to N. On qubits alone, this is the backward light cone of W. The classical bits extend it, from a measurement to the operations its bit conditions. Take the program of Fig. 1 with a one-bit secret, the circuit

$$P_1 \;=\; \mathrm{H}(q_1)\;\; \mathrm{Z}(q_1)\;\; \mathrm{H}(q_0)\;\; \mathrm{CX}(q_0, q_1)\;\; \mathrm{H}(q_0)$$

whose gates are those of lines 24, 25, 29, 40 and 47, and the criterion $\{q_0\}$. Walking backward, the last $\mathrm{H}(q_0)$ is kept, the CNOT acts on q_0, so it is kept and q_1 joins N, and then $\mathrm{H}(q_0)$ and the two gates on q_1 are kept as well, so the slice is the whole circuit. The gates on q_1 prepare it in $|-\rangle$, and through phase kickback they turn q_0 into $|1\rangle$, although q_0 is the CNOT's control. A rule that reads the control as unchanged lets the CNOT write only q_1, so its slice keeps the two Hadamard gates on q_0 alone, lines 29 and 47 of Fig. 1.

On qubits, the soundness of this slice is a known property of the light cone, which compilers and analyses of variational circuits rely on [10], [39]. We state it for the QDG, classical bits included, and prove it, because the results that follow build on its proof.

Theorem 1 (The QDG is sound): For every circuit P and criterion W, $B_{\mathrm{QDG}}(W)$ is a sound slice for W.

Proof sketch: Process P backward with the needed set N, initially W. Operation o_k joins the slice, and N grows by $loc(o_k)$, exactly when $loc(o_k) \cap N \neq \emptyset$, which computes $B_{\mathrm{QDG}}(W)$. Every gate, reset, measurement and conditioned gate is a channel that acts only on $loc(o)$. By locality of the

partial trace, the no-signaling principle, an operation outside the slice cannot change the reduced state of the needed locations after it, and an operation in the slice maps the reduced state of the needed set before it to the one after it. Induction over P gives the claim. The induction is mechanized in Isabelle/HOL for an abstract model of local operations, and that quantum operations satisfy the model is argued on paper. ∎

Proposition 1 (The QDG is dependence-complete): $Dep_W(P) \subseteq B_{\mathrm{QDG}}(W)$.

Proof: Let $o_k \notin B_{\mathrm{QDG}}(W)$, and let P' insert a unitary U after o_k. In the backward pass, no location of o_k is needed after it, since otherwise o_k would be kept. U acts only on locations of o_k, so U is not kept either and the needed set is unchanged. Hence P' and P have the same QDG slice, and by Theorem 1 the same reduced state of W. A negated outcome is handled alike, as an operation on the bit of a measurement outside the slice. ∎

Proposition 2 (The QDG distributes over criteria): $B_{\mathrm{QDG}}(W_1 \cup W_2) = B_{\mathrm{QDG}}(W_1) \cup B_{\mathrm{QDG}}(W_2)$.

Proof: In the backward pass of Theorem 1, the needed set for $W_1 \cup W_2$ is at every position the union of those for W_1 and W_2. ∎

Together these say that the QDG slice keeps everything a slice must keep and reproduces the criterion for every input, and that the slice of a set of qubits is the union of the slices of its members, so a tool computes single-qubit slices and combines them. It is not always the smallest such set, since an operation can touch a needed qubit without changing it, and Section VI measures how close it comes to the dependences themselves. What is new is not the relation but what it is shown to be, a slice that keeps every dependence, against which every other rule can be measured.

Why the write set cannot be smaller: Dropping a qubit from the write set of a two-qubit gate is safe only if that gate can never carry information from the other qubit to it. No standard gate qualifies. For each of the 21 two-qubit gates of Qiskit's standard library, from the CNOT and CZ to the SWAP and the parameterized rotations, a change of either qubit's input changes the reduced state of the other for some product input, an exploratory check of Section V. A rule that reads any of these gates as writing only some of its qubits therefore misses a dependence of some circuit.

C. Three kinds of dependence suffice

The write-all rule needs only three kinds of dependence, which Table II lists with examples from the ansatz of Fig. 2. *Classical data* dependence follows the host values that compute gate arguments, such as angles, sizes and qubit indices. *Classical control* dependence follows what decides whether an operation is applied at all. It covers the host program's loops and branches, which shape the circuit as it is built, and the measured bits that condition operations when the circuit runs, in a conditioned gate or a branch of the circuit. *Quantum data* dependence follows a qubit through the operations that act on it, under the write-all rule.

TABLE II
THE THREE KINDS OF DEPENDENCE A QUANTUM SLICE NEEDS

Kind	Follows	In Fig. 2a
Classical data	host values that compute gate arguments	`n`, `layers`, the angles
Classical control	host loops and branches, and measured bits that condition operations	`for qubit in range(n)`, the flip conditioned on the measured bit
Quantum data	every qubit an operation acts on, written by the write-all rule	`cx(qubit, qubit + 1)` writes both qubits

A controlled gate adds no fourth kind. Its control is written like its target, so what looks like quantum control dependence is quantum data dependence.

A controlled gate looks like a fourth kind, quantum control dependence, since it reads like a branch, if the control is 1 then flip the target. It is not one. Its control is coherent, so the gate acts on a superposition of control values and no branch is ever taken, and it can change its control, as P_1 shows. What looks like control dependence is quantum data dependence in both directions, which the write-all rule records, and reading the gate as a branch is exactly the error of the classical PDG in Section IV-D. Within the circuit, control dependence arises only through measurement, whose classical bit then conditions later operations.

The other kinds proposed for quantum programs add no dependence either. Entanglement is the most natural candidate, since entangled qubits are correlated, but it decides nothing.

Proposition 3 (Entanglement is neither necessary nor sufficient for dependence): (i) An operation can be a dependence of q although, at an input that witnesses the dependence, q is never entangled with its qubits. (ii) An operation can act on a qubit entangled with q and not be a dependence of q.

Proof: (i) In P_1 at input $|00\rangle$, the state is a product state after every operation, yet the gates on q_1 are dependences of q_0, since inserting $\mathrm{Z}(q_1)$ after them changes the final state of q_0. (ii) In $\mathrm{H}(q_0)\,\mathrm{CX}(q_0, q_1)\,\mathrm{X}(q_1)$ the two qubits are maximally entangled before $\mathrm{X}(q_1)$, but by no-signaling no operation on q_1 after the CNOT can change ρ_{q_0}. ∎

By Proposition 1, entanglement edges can add no dependence that the QDG misses, and by (ii) they can add operations that are not dependences. The remaining proposed kinds add no edges. A CNOT that copies a basis state, the case no-cloning concerns, is a two-qubit gate whose edges quantum data dependence already has. Timing decides when an operation runs on hardware, not what a state depends on [43]. Superposition changes amplitudes, not which locations an operation acts on [57], so it cannot add an edge to a dependence graph.

D. The other rules, relative to the write-all rule

The study compares the write-all rule with the rules a developer or tool builder would otherwise use. Table III gives each by how it differs from the QDG, and Fig. 2 applies them to one program that needs every kind of dependence. An

TABLE III
THE DEPENDENCE RULES, BY WHAT A CNOT WITH CONTROL c AND TARGET t WRITES AND BY HOW EACH DIFFERS FROM THE QDG

Rule	$w(cx_{c,t})$	**Relation to the QDG**	**Guarantee**	**Ops**	F
QDG	$\{c,t\}$	**write-all rule**	**sound, complete**	16	1.00
Def-use	$\{t\}$	− control write, − bits	none	5	0.55
Classical PDG	$\{t\}$	− control write	none	7	0.97
QPDG [7]	$\{c,t\}$	+ closed entanglement edges	sound	19	1.00
QPDG, read-only	$\{t\}$	− control write, + entanglement	none	19	1.00

Every rule reads $\{c,t\}$, and complete means dependence-complete. Commutation and state-based pruning remove operations from the QDG slice. Ops is the number of the 19 operations of Fig. 2 that the slice of q_0 keeps, and F its fidelity at $|0\dots0\rangle$.

ansatz is the parameterized circuit of a variational algorithm, whose rotation angles a classical optimizer tunes. The ansatz of Fig. 2a has layers of rotations and a chain of CNOTs, and after them an auxiliary qubit is measured mid-circuit and q_0 is flipped when the outcome is 1.

The classical PDG and def-use: The classical PDG, written CPDG in subscripts, applies the program dependence graph [14] to the circuit. It reads a gate as a classical assignment, so a controlled gate reads its controls and targets and writes only its targets, and any other gate reads and writes all its qubits. A measurement writes its bit, and a conditioned gate reads the bits its condition tests. *Def-use* (DU) is the same data dependence without the bits, the simplest transfer of classical slicing. Reading a control as unchanged is the usual explanation of a CNOT [29], and it holds for computational basis states.

Proposition 4 (The classical PDG is unsound): There are circuits for which neither $B_{\mathrm{CPDG}}(W)$ nor $B_{\mathrm{DU}}(W)$ preserves W, even at $|0\dots0\rangle$.

The ansatz of Fig. 2 is a witness for both, and on unitary circuits the two slices are equal. Def-use ignores the classical bits, so its slice of q_0 keeps only 5 of the 19 operations, with fidelity 0.55, and loses the auxiliary's gate and measurement, which the classical PDG reaches through the green control edge in Fig. 2b. Control dependence is not enough, however. Each `cx(qubit, qubit + 1)` changes its control as well as its target, so the state of q_0 comes to depend on the rotations of q_1 and, from the second layer on, of q_2. The classical PDG keeps only 7 operations, misses 9 dependences and reaches fidelity 0.97. Where the QDG has a blue quantum edge out of a CNOT, the classical PDG has a dotted edge that skips the CNOT. The QDG slice keeps the 16 operations that are dependences of q_0, and no other.

The QPDG: Our reimplementation of Yu and Zhao's graph [7] serves as a baseline. Its dependences are given in prose, so we logged every reading of it before any run. Data and control dependence are as in the QDG. The QPDG gives no rule for when two qubits are entangled, so we use the flow-insensitive rule of entanglement analysis, under which qubits joined by a two-qubit gate anywhere in the circuit are entangled [37], and add an edge from every operation to every later one on qubits of the same component. A flow-sensitive variant counts two qubits as entangled only from the first gate that couples them. Either way the QPDG is the QDG plus entanglement edges, so its slices contain those of the QDG, and in Fig. 2b the extra edges add the three hatched operations of q_2 in the second layer, which run after the last gate that carries information from q_2 to q_0. Yu and Zhao do not say whether a gate writes its control, so we also run, as an exploratory variant, the reading in which it does not.

Pruning: Two refinements start from the QDG slice and remove operations from it. *Commutation pruning* propagates the Pauli observables X, Y and Z of every criterion qubit backward and removes each operation that commutes with all of them, since such an operation cannot change a measurement statistic of the criterion. Qiskit's `LightCone` pass implements it [11]. *State-based pruning* fixes one input, by default $|0\dots0\rangle$, and removes an operation that does not act on the criterion if the mutual information between the criterion and the operation's qubits is below 10^{-9} and the removal still preserves W at that input. Both give smaller slices, but neither is dependence-complete, and a state-based slice holds for its one input only, as a conditioned slice does [58].

E. Forward slices and source lines

The forward slice $F_{\mathrm{QDG}}(o_k)$ of an operation is the set of operations reachable from o_k in the QDG. It answers the question a developer asks before changing a gate, which later operations and qubits the change can reach. We say that o_k reaches q if $F_{\mathrm{QDG}}(o_k)$ contains the last operation on q.

Proposition 5 (Forward and backward slices are dual): For every operation o_k and qubit q, o_k reaches q if and only if $o_k \in B_{\mathrm{QDG}}(\{q\})$. Hence o_k reaches every qubit of which it is a dependence.

Proof: Both sides say that the QDG has a path from o_k to the last operation on q. The second claim follows from Proposition 1. ∎

A slice can be reported as operations, loop iterations or source lines. Operations are the unit of the definitions and lines the unit of the developer. A line is in a slice if one of its operations is, so a single iteration can put a line in a slice. In the second layer of the ansatz, the rotations of q_2 are not dependences of q_0, yet their lines are in its slice because their first-layer iterations are. A line slice therefore names the iterations that put each line in it.

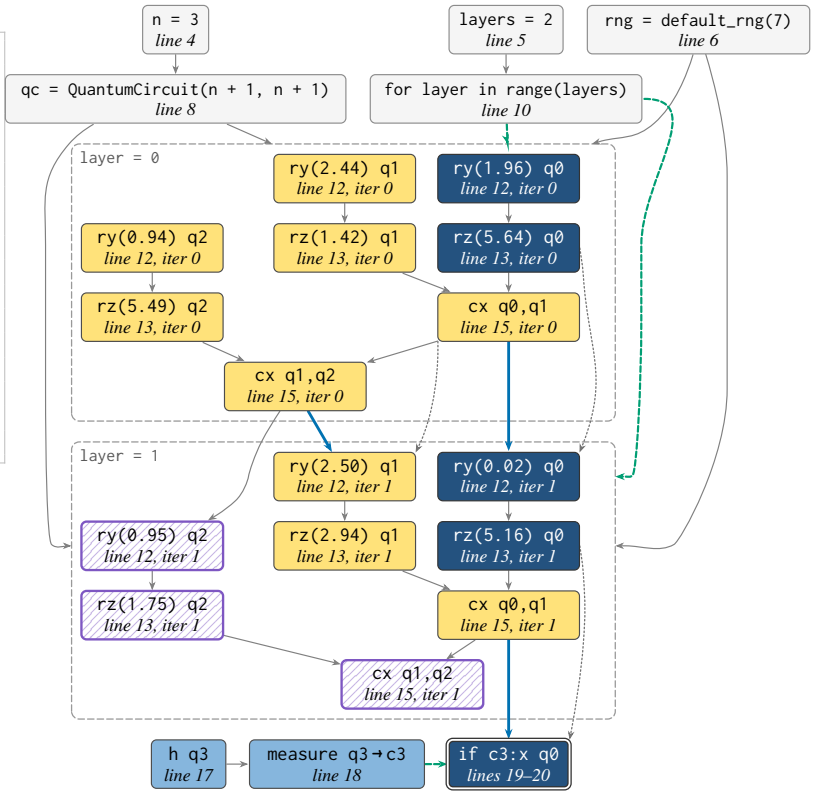
A slice is computed on the circuit, and a developer reads it on the program. Every operation records the source line and the loop iterations that appended it, so a slice of operations maps to lines, and the host code that computes the kept operations' qubits, angles and loop bounds follows by a classical slice of the host program [1]. The record can be taken while the program builds its circuit, as the study does for Bugs4Q, or by a static analysis of the source, as Behler

(a) VQE ansatz

```
4  n = 3
5  layers = 2
6  rng = np.random.default_rng(7)
7
8  qc = QuantumCircuit(n + 1, n + 1)
9
10 for layer in range(layers):
11     for qubit in range(n):
12         qc.ry(float(rng.uniform(0, np.pi)), qubit)
13         qc.rz(float(rng.uniform(0, 2 * np.pi)), qubit)
14     for qubit in range(n - 1):
15         qc.cx(qubit, qubit + 1)
16
17 qc.h(n)
18 qc.measure(n, n)
19 with qc.if_test((qc.clbits[n], 1)):
20     qc.x(0)
```

data
control (measured bit or loop)
quantum, not in the classical PDG
classical PDG and def-use only, skips a CNOT
kept by every rule, def-use included
added by the classical PDG
added by the QDG
added by the QPDG, not a dependence
last write to the criterion
classical statement, all in the QDG slice
one iteration of the outer loop, with the inner loops of lines 11 and 14 unrolled

(b) Dependence graph of the four rules, criterion q_0



slices of q_0, of 19 operations:
def-use 5, $F = 0.55$ · classical PDG 7, $F = 0.97$
QDG 16, exactly the dependences, $F = 1.00$ · QPDG 19, $F = 1.00$

Fig. 2. The rules on a variational ansatz, criterion q_0. (a) The program, described in Section IV-D. (b) One graph for the first four rules of Table III. Each node is an operation as it executes, with its source line and loop iteration, and a box holds each iteration of the outer loop. Grey data edges belong to every rule, and def-use lacks the green control edge from the measurement. The classical PDG and def-use lack the blue quantum edges and have the dotted edges instead, each of which skips a CNOT whose control it reads as unchanged. A node is shaded by the weakest rule that keeps it. The QDG keeps exactly the dependences of q_0, and the QPDG adds the hatched operations, which are not dependences. Graphs, slices and fidelities at $|0\ldots0\rangle$ are computed by the study's code from the program shown.

et al. derive the order of qubit operations from OpenQASM 3 source [28]. Where a value is unknown statically, such an analysis must assume every qubit an expression may denote and every branch a condition may take, which can only enlarge the slice. The definitions of this section are stated on the circuit and hold for any front end that produces it.

V. Study Design

The study measures every rule of Section IV against the definitions of that section. It asks four research questions, each a question a developer has of a slice, whether it can be trusted, how small it can safely be, whether it holds the fault, and what it costs.

- **RQ1** Which dependence rules give sound and dependence-complete slices?
- **RQ2** How much smaller than the QDG slice can a slice safely be, and under which inputs does the extra precision hold?
- **RQ3** What does each rule do to fault localization, measured by whether the slice contains the fault and by mutation-based fault localization?
- **RQ4** What does each rule cost in time as circuits grow?

A. Pre-registration

The protocol was committed before the runs reported here. It extends an earlier run whose results it reuses, so the outcomes of H1 and H2 below were known when it was written. Every later change is a dated deviation, marked when it was made after a result. The five confirmatory tests concern state-based pruning. Every other result, including the classical PDG's share of wrong slices, is exploratory or descriptive, and so is everything added after the first results. These additions are the classical PDG and `LightCone` baselines, the dependence set as a size reference and its nesting with the rules, the symbolic-angle, flow-sensitive and read-only variants, and the check of two-qubit gates in both directions. The protocol also pre-registered two descriptive tasks for language models, from

TABLE IV
CIRCUITS OF THE STUDY

Subject	Circuits	Qubits	Operations	Used for
Corpus A	158	1–14	14 / 126	RQ1 to RQ4
Generated circuits	119	2–10	19 / 2,119	RQ1, RQ4
Width sweeps	22	4–14	26 / 104	RQ4
Corpus L	55	15–20	120 / 783	RQ1, RQ4

Qubits are given as smallest to largest, operations as median / largest. Faults are seeded into corpus A, giving 5,688 units from 2,662 mutants, and seven of the Bugs4Q bugs give 15 units. A unit is a fault and a qubit whose measured distribution it changes.

which the most expensive model was dropped for cost before any scored call.

B. Subjects

The rules are compared on circuits, where an exact simulation gives the ground truth, and Table IV lists them. Corpus A collects benchmark circuits of at most 14 qubits from MQT Bench [59], QASMBench [60] and VeriQBench [61]. Every qubit of every circuit is a criterion, which gives 784 circuit–qubit pairs, 80 of them in dynamic circuits with mid-circuit measurement, reset or conditioned gates. The generated circuits come in families of known structure that grow in depth and in width, the width sweeps extend two of the families, and corpus L, drawn from the same three suites, has wider circuits on which only some rules finish. Real faults come from Bugs4Q [12], as the next subsections describe.

C. Ground truth

An independent oracle simulates each circuit exactly, branching over measurement outcomes, and agrees with Qiskit's statevector simulation to within 2.2×10^{-16}. A slice preserves the criterion when the squared Uhlmann fidelity between the criterion's reduced states under the slice and under the whole circuit is at least $1 - 10^{-6}$. The dependences of Definition 3 are measured by inserting two random unitaries after every operation and comparing the criterion's state at 11 inputs, $|0 \ldots 0\rangle$ and random product states. Because the inputs and perturbations are sampled, the measured dependence set can only miss true dependences, never add false ones. A catalog of 24 hand-derived cases checks the ground truth and every slicer before the runs.

D. Slicers

Each rule of Section IV-D is implemented on circuits from its read and write sets or its pruning step. Commutation pruning uses Qiskit's `LightCone` pass, which applies to 732 of the 784 pairs. It rejects the circuits with mid-circuit measurements, 42 pairs, and on 10 it returns a circuit whose operations cannot be matched back to the original in order. Exploratory variants make commutation pruning hold for all angles and give state-based pruning a joint criterion.

E. Robustness

A slice computed at $|0 \ldots 0\rangle$ is rechecked, with its operations unchanged, under 100 random computational-basis inputs, 100 random product inputs and, for circuits with rotation angles, 100 random rebindings of every angle. A pair is robust if the slice preserves the criterion in every draw.

Whether a two-qubit gate can carry information from either of its qubits to the other is checked on each of the 21 two-qubit gates of Qiskit's standard library, parameterized gates at a fixed angle. For each direction, seeded random product inputs are searched for two inputs of one qubit that leave the other qubit in different reduced states. This is evidence for these gates, not a proof for all gates.

F. Faults and fault localization

Faults are seeded with ten operators, nine from Muskit [51], QMutPy [52] and García de la Barrera et al. [53], and an angle perturbation of our own. Of the 5,375 mutants, 4 with an empty fault site are excluded, and 2,662 change the measured distribution of at least one qubit. Each such qubit and its mutant form a unit, and for each unit we record the slice's size and whether it contains the fault site. Of the 107 entries of Bugs4Q, 11 are circuit-level faults that port mechanically to the current Qiskit release, and seven of them change a qubit's measured distribution. Running each port records, for every operation, the line that appends it, which is mapped to the line of the original program, so the slices of these units are also reported as lines of the programs their developers wrote.

Mutation-based fault localization ranks the operations of the faulty circuit by how their mutants change its observable behavior, with the Ochiai formula, as Ishimoto et al. do for quantum programs [54]. It runs on a sample of 293 units, stratified by fault operator and including every Bugs4Q unit. We rank over the whole circuit and within each slice and report the EXAM score, the share of ranked operations examined before the fault, lower being better. Without fault localization, we also order each slice from its last operation backward and score the fault's rank the same way. A slice never makes a developer examine more operations, but EXAM divides by the number ranked, so a smaller slice can raise it.

The two tasks for language models ask what a slice offers a model that reads a circuit. In the first, a model lists the operations that can affect a qubit, for 171 items. In the second, it names the likely faulty operation of 276 units, given the whole circuit or a slice. The models are Claude Sonnet 5.5 and Haiku 5.5 and the open Qwen2.5-Coder models with 32 and 1.5 billion parameters.

G. Cost

RQ4 times the rules on the generated family that grows in depth at a width of eight qubits, on width sweeps of two generated families from 4 to 14 qubits, and on corpus L, each circuit in a fresh process with limits of 120 s and 4 GiB.

TABLE V
THE QDG ON EVERY QUESTION OF THE STUDY

Question	Expected	Observed
Sound (RQ1)	every input, Theorem 1	0 wrong slices of 784 pairs, and of 633 on corpus L
Complete (RQ1)	Proposition 1	recall 1.000 of the measured dependences
Size (RQ1)	not minimal	4.8 points above the dependence set, equal to it on 559 pairs
Noise (RQ2)	local noise, Theorem 1	trace distance at most 4.2×10^{-15}
Faults (RQ3)	no guarantee	contains 95.7% of the faults, 17.5 points above a random set of its size
Cost (RQ4)	one pass	at most 7 ms per criterion

TABLE VI
SLICES OF THE 784 CORPUS-A PAIRS FOR RQ1

Rule	Wrong	Dynamic	Recall	Precision	Size
Dependence set[a]	–	–	1.000	1.000	65.6%
QDG	**0**	0	**1.000**	0.923	70.4%
Def-use	216	13	0.718	0.970	51.2%
Classical PDG	204	1	0.741	0.962	53.0%
QPDG [7]	0	0	1.000	0.835	78.6%
QPDG, flow-sensitive[b]	0	0	1.000	0.849	77.5%
Commutation[c]	0	0	0.930	0.967	64.4%
Commutation, symbolic[c]	0	0	0.939	0.966	65.2%
State-based	0	0	0.807	0.972	54.0%

Wrong counts the slices that do not preserve the criterion at $|0\ldots0\rangle$, a lower bound for every rule but the QDG, which is sound for all inputs, and Dynamic those among the 80 pairs in dynamic circuits. Recall and precision are against the dependence set, and size is the mean share of the circuit. [a] The smallest slice that contains every dependence, a reference and not a rule. [b] Exploratory, the variant of Section IV-D. [c] On the 732 pairs where Qiskit's pass applies, 38 of them dynamic.

H. Statistics

Five confirmatory hypotheses compare state-based pruning with the other rules. H1 states that it is wrong on fewer pairs than def-use, H2 that its slices are smaller than QDG slices, and H3 that at least 95% of pairs keep a correct slice under random product inputs. H4 states that it loses at most five percentage points of fault containment against the QDG slice, and H5 that its slices are smaller where both contain the fault. State-based pruning starts from the QDG slice and checks every removal at $|0\ldots0\rangle$, so H1 and the directions of H2 and H5 hold by construction, and H3 and H4 are the real tests. We use exact McNemar tests on paired outcomes and Wilcoxon signed-rank tests on paired sizes, with Holm's correction across the five. Effect sizes come with bootstrap 95% intervals and Wilson intervals for proportions, and exploratory intervals resample whole algorithm families. Everything else is descriptive.

VI. RESULTS

Table V gives the answer first. The QDG does what Section IV predicts on every pair the oracle can check, comes within a few points of the smallest complete slice, holds under noise, and costs milliseconds. It is not guaranteed to contain a fault, since a fault can delete the very interaction a slice would follow, and RQ3 measures how often that happens. The subsections answer each question for every rule.

A. RQ1: Which rules give sound and complete slices?

One choice decides soundness, what a controlled gate writes. The classical PDG, which lets a CNOT write only its target, misses 3,174 dependences over the 784 pairs, and all but 10 of them, 99.7%, follow from reading a control as unchanged. Of the misses, 822 are controlled gates whose control is the criterion qubit, 1,688 reach the criterion only through such a gate, 654 reach it through a control further along the chain, and only 10 pass through classical feedback. As a result, Table VI shows, the classical PDG gives a wrong slice for 204 of the pairs, or 26.0%, with a 95% interval of 17% to 35% when the 93 algorithm families are resampled as clusters. Its control dependence matters only on dynamic circuits, where it is wrong on 1 pair against 13 for def-use.

Many of these wrong slices look right. Of the 204, 88, or 43%, give the criterion the same distribution of outcomes in the computational basis as the whole circuit, so only 116 pairs, 14.8% of all, are wrong in what a terminal measurement reads. The others are wrong in phase, which a histogram check accepts but which changes the outcome once the slice is composed with later gates or measured in another basis. Testing meets the same limit, since phase-flip faults escape measurement in the computational basis [47]. A developer who checks a slice by its outcomes would therefore accept almost half of the wrong ones. As H1 expects by construction, state-based pruning, which starts from the QDG slice, is wrong on fewer pairs than def-use, 0 against 216 (exact McNemar, Holm $p < 10^{-10}$).

The QDG is sound on every pair and recovers every measured dependence, as Theorem 1 and Proposition 1 predict. The QPDG is sound too, since its data edges let every gate write all its qubits, but its entanglement edges add only operations that are not dependences, so its precision falls from 0.923 to 0.835 and its slices grow from 70.4% to 78.6% of the circuit. The flow-sensitive variant still keeps 11.9 percentage points above the dependence set, so the excess comes from the entanglement edges and not from our reading of them. A graph with the QPDG's typed edges and one entanglement edge per two-qubit gate gives exactly the QDG slice on all 779 pairs it applies to and differs only when these edges are closed transitively, so the per-gate edge is the write-all rule and the closure is what costs precision. Under the other reading of Yu and Zhao, in which a controlled gate writes only its targets, the QPDG is wrong on 10 pairs from 3 circuits and misses 502 dependences, 314 of them by kickback. On the 633 pairs of the 39 corpus-L circuits that the oracle can check exactly, the QDG and the QPDG are again sound everywhere, while

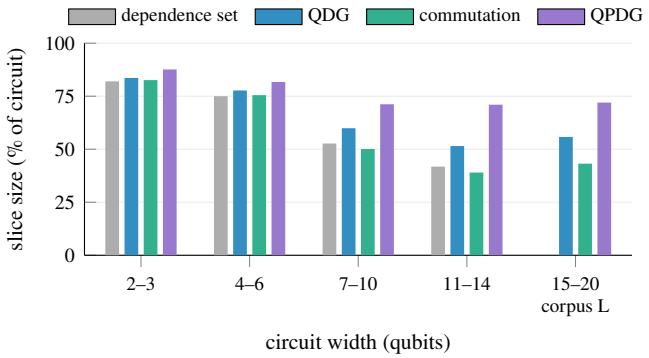


Fig. 3. Mean slice size by circuit width, for RQ1. The dependence set is the floor that every complete slice contains. Corpus L has no dependence set, since measuring one there would need more simulation than the oracle can do exactly.

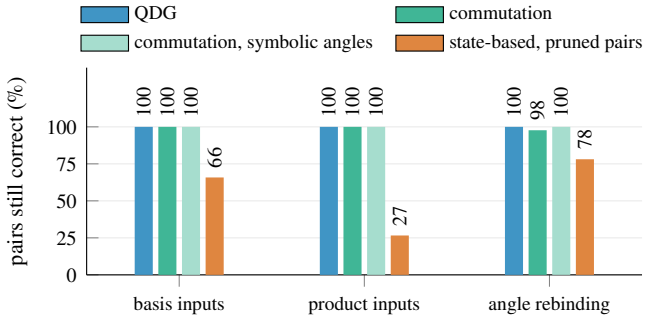


Fig. 4. Share of pairs whose slice, computed at $|0\ldots0\rangle$, stays correct in all 100 draws of each input regime, for RQ2. Commutation is shown on the 732 pairs where Qiskit's pass applies, state-based pruning on the 477 pairs it prunes, and angle rebinding on the pairs of circuits with rotation angles.

def-use is wrong on 205.

The rules nest, and Fig. 3 shows how far apart they are. On all 784 pairs the dependence set lies within the QDG slice and the QDG slice within the QPDG slice. Restricted to its dependence set, every circuit preserves the criterion at $|0\ldots0\rangle$, so the dependence set is itself a sound slice and the smallest one that contains every dependence. It makes up 65.6% of a circuit on average. The QDG slice keeps only 4.8 percentage points more and equals it on 559 of the 784 pairs, while the QPDG keeps 13.0 points more, 2.7 times as much code that cannot change the criterion. The gap grows with width. At 11 to 14 qubits the QDG slice keeps 51.5% of the circuit and on corpus L 55.8%, while the QPDG stays near 72.0%, because it links the operations of a whole coupling component. Commutation pruning falls below the dependence set, to 39.0% at the same widths, because it is sound for fixed angles but not complete. It can drop an operation where a fault would change the criterion, as the identity gate of line 38 in Fig. 1 shows, which is why completeness, not size, is what a developer needs from a slice.

TABLE VII
FAULTS AND SLICES ON THE 5,688 SEEDED-FAULT UNITS, FOR RQ3

Slice	Contains	Size	Random	Gain	EXAM
Whole circuit	100%	100%	–	–	0.320
QDG	95.7%	76.8%	78.2%	17.5	0.394
QPDG	97.4%	82.9%	84.0%	13.4	0.369
Classical PDG	78.8%	59.0%	60.7%	18.1	0.470
Def-use	77.3%	57.5%	59.2%	18.1	0.471
State-based	82.2%	60.5%	62.4%	19.8	0.440

Contains is the share of units whose slice of the faulty circuit contains the fault site, size the mean share of the circuit, Random the containment of a random set of the slice's size, and Gain the difference in percentage points. EXAM is the mean score of mutation-based fault localization of Section V, lower being better, on the 223 sampled units whose fault every slice contains.

B. RQ2: How much precision beyond the QDG slice is safe?

Both refinements make slices smaller, but only commutation pruning keeps them correct for fixed angles. State-based pruning shrinks the QDG slice by 16.3 percentage points of the circuit (95% CI 15.1 to 17.7) and is smaller on 477 pairs (H2, Wilcoxon, Holm $p < 10^{-10}$). Fig. 4 shows what that costs. Of the pairs it prunes, the slice stays correct for all random product inputs on only 26.6%, and over all pairs on 55.4% (Wilson CI 51.9% to 58.8%, 41% to 68% with families as clusters), far below the pre-registered threshold of 95%, so H3 fails. A state-based slice is a property of one input, not of the program.

Commutation pruning is smaller than the QDG slice on 243 of 732 pairs, stays correct on every random basis and product input, and fails only when the angles change, on 4 of 175 pairs of variational circuits. Its symbolic variant, at a mean size of 65.2% against 64.4%, stays correct on every pair in every regime. Its recall of 0.930 shows that it misses dependences, since a different gate in place of one it drops could change the criterion.

Under depolarizing noise at two-qubit error rates of 10^{-3} to $3\cdot10^{-2}$, in a pre-registered descriptive analysis, the criterion's noisy states under the QDG slice and the whole circuit differ in trace distance by at most 4.2×10^{-15} where the simulation is exact and 0.033 where it samples, which is rounding and sampling error, as Theorem 1 predicts for noise local to a gate. The state-based slice differs in 702 of 1,746 exact runs, by up to 0.24, because it removes noisy gates that the whole program executes [62], so it can hide the noise a developer is trying to diagnose.

C. RQ3: What does each rule do to fault localization?

Table VII shows that a slice that contains the fault more often is also larger, so containment has to be read against a random set of the slice's size. The QDG slice contains the fault in 95.7% of the units and the QPDG slice in 97.4%, but a random set of the QPDG's size already contains it in 84.0%, and Section VI-E returns to this comparison. Against a random set of its own size, every rule contains the fault more often, the QDG by 17.5 points (family CI 13.2 to 21.8), so containment

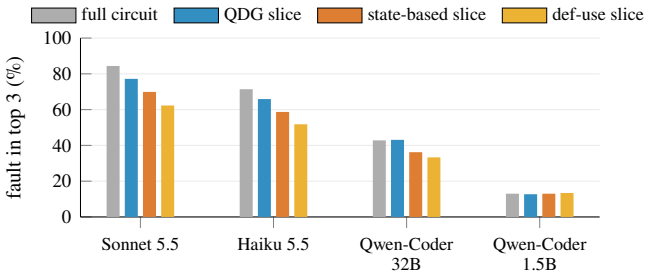


Fig. 5. Share of the 276 units whose faulty operation a language model names among its top three, given the whole circuit or a slice, for RQ3.

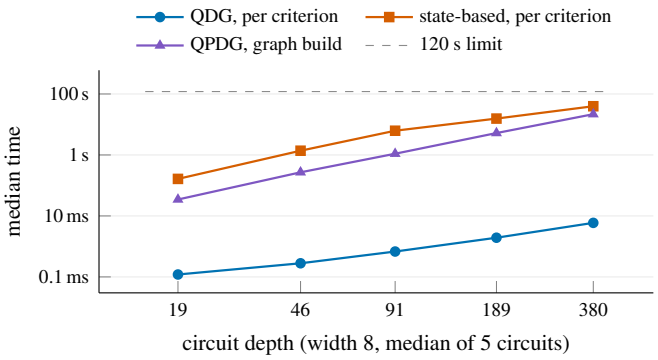


Fig. 6. Median time against circuit depth at a width of 8 qubits, over five circuits per depth, for RQ4. Runs beyond the 120 s limit are left out.

is not only a matter of size. All 245 units whose fault the QDG slice misses are omission faults, a removed control (66), a retargeted operand (91), a removed preparation of an entangling gate (59) or a deleted gate (29). Each deletes the interaction that the slice of the faulty program would follow, so the place of the missing operation is not in the slice. The QPDG contains 96 of them only because its closure still links operations that the faulty program no longer connects.

State-based pruning lowers containment by 13.5 percentage points against the QDG slice (95% CI -14.4 to -12.6, -19.9 to -7.5 with families as clusters), so H4, which allowed a loss of at most five points, fails. In 769 units the QDG slice contains the fault and the state-based slice does not. Pruning removes an operation when the criterion is uncorrelated with its qubits at the input, so a fault that breaks a correlation can make the faulty operation look irrelevant to the single criterion qubit, and a joint criterion over all wrong qubits recovers 691 of these units. Where both slices contain the fault, the state-based slice is smaller by 2.9 operations on average, so H5 holds, but its EXAM score is higher by 0.061.

Slices do not improve mutation-based fault localization. On the 223 units whose fault every slice contains, ranking within any slice reaches the fault after 1.8 to 1.9 operations on average, as ranking over the whole circuit does, and the EXAM score, which divides by the number of operations ranked, is lowest for the whole circuit. A slice narrows where to look, but it does not reorder what mutation analysis finds.

On the 15 real Bugs4Q units whose faulty lines map to the program, the QDG line slices keep every faulty line and 10.3 of 10.7 lines on average, so on these small programs slicing barely reduces what there is to read. Def-use, which equals the classical PDG on these unitary programs, loses the faulty line in one of the units.

Language models: In the first task of Section V, Claude Sonnet 5.5 and Haiku 5.5 recall 0.84 and 0.84 of the dependences at precision 0.93 and 0.94, and both Qwen models recall 0.59, where the QDG slice recalls them all. The models differ most on the dependences through kickback, recalled from 0.16 by the larger Qwen model to 0.83 by Haiku, and 0.61 by Sonnet. In the second task, Fig. 5 shows that no slice helps a model localize a fault. Sonnet has the fault in its top three for 84.4% of the units with the whole circuit, 77.2% with the QDG slice and 69.9% with the state-based slice, 7.2 points below the QDG slice (95% CI $[-12.3, -2.5]$, exact McNemar $p = 0.006$), Haiku shows the same order, and for every model no slice improves on the whole circuit by more than a fraction of a point. A model thus misses the dependences a slicer finds by rule, yet a slice does not help it localize a fault, and a pruned slice hurts it.

TABLE VIII
THE QDG AND THE QPDG ON EVERY MEASURE OF THE STUDY

Measure	QDG	QPDG	RQ
Wrong slices	0	0	RQ1
Dependences missed	0	0	RQ1
Mean slice size	70.4%	78.6%	RQ1
Code beyond the dependences	4.8 points	13.0 points	RQ1
Pairs equal to the dependence set	559	426	RQ1
Precision	0.923	0.835	RQ1
Units whose fault the slice contains	95.7%	97.4%	RQ3
Gain over a random set of its size	17.5 points	13.4 points	RQ3
EXAM of fault localization	0.394	0.369	RQ3
Time at depth 380	≤ 7 ms	21.7 s	RQ4

Shading marks the better value. Time is per criterion for the QDG and the median graph build for the QPDG.

D. RQ4: What does it cost?

The QDG slice costs at most 7 ms per criterion at every depth, as Fig. 6 shows. State-based pruning, which simulates the circuit for every candidate removal, exceeds the limit of 120 s for four of the five circuits at the second-largest depth and for all five at the largest, grows exponentially with width, and completes on 39 of the 55 circuits of corpus L. Building the QPDG takes 1.10 s at depth 91 and 21.7 s at depth 380 in the median, against milliseconds for the QDG slice, because its closure links every pair of operations on a coupling component while the QDG needs only the operations on each qubit.

E. The QDG against the QPDG

The QPDG looks as good as the QDG on correctness and better on fault localization, and Table VIII shows why it is not. Both are sound and both contain every dependence, since the QPDG is the QDG with entanglement edges added. Everything those edges add is code that cannot change the criterion, 13.0 points of the circuit on average against 4.8 for the QDG, and on 287 of the 784 pairs the QPDG slice is larger than the QDG slice. A developer reading a QPDG slice reads 2.7 times as much irrelevant code.

The QPDG's two wins come from that extra code. It contains the fault in 97.4% of the units, but a random set of its size already does in 84.0%, so per unit of size the QDG locates faults better, by 17.5 points over random against 13.4. The 96 units the QPDG catches and the QDG misses are omission faults, whose missing operation the closure happens to link, not by a guarantee. Its lower EXAM score is a size effect as well, since EXAM divides by the number of operations ranked, and the whole circuit has the lowest score of all. The costs are not close. The QDG slices a criterion in at most 7 ms at every depth, while building the QPDG takes 21.7 s at depth 380. Under the reading in which a control is left unchanged, which Yu and Zhao's prose allows, the QPDG is not even sound and is wrong on 10 pairs. For comprehension the QDG is the better rule. It keeps everything that can matter, adds the least that cannot, and is computed in milliseconds.

VII. Implications for Comprehension Tools

The results turn into advice for those who build slicers and other comprehension aids for quantum programs, for the developers who read their output, and for research on both.

What a slicer must do: A slicer must let every gate write all its qubits. Reading the control of a gate as unchanged is the one choice behind 99.7% of the dependences the classical PDG misses in RQ1, and it is the reading that the QCPG states and that an order table invites when it marks the control apart from the target. A representation built for pattern detection can keep that distinction for reporting, but a slicer that reads it as a dependence graph will hide code that can change the qubit. A slicer must also follow measured bits to the operations they condition, since without them def-use is wrong on 13 of the 80 pairs in dynamic circuits. It should not add entanglement edges, which add only code that cannot change the criterion, 2.7 times as much as the QDG keeps beyond the dependences, and turn a pass of milliseconds into a graph that takes 21.7 s to build at the largest depth of RQ4. Because a single loop iteration can put a line in a slice, a slicer should show the iterations that put each line there, so that the developer does not read a whole loop body as relevant.

What a developer should not trust: A slice cannot be validated by its outcomes. Of the wrong classical slices, 43% reproduce the outcome histogram of the whole program, and the Bugs4Q example of Section III shows that a fault can leave every single-qubit histogram unchanged. A check that compares histograms accepts exactly the slices that are wrong in phase or in correlation, so a tool that offers such a check should compare the criterion's reduced state, or at least its joint outcomes in more than one basis. Pruned slices answer a narrower question than the one a developer asks. A slice pruned by commutation is correct for the observable at the circuit's angles, and Qiskit's pass is the right tool to estimate that observable, but it drops dependences, recalling 0.930 of them, and it fails on 4 of 175 variational pairs when the angles change. A state-based slice is correct for its one input, stays correct for random inputs on only 55.4% of the pairs, contains the fault 13.5 points less often than the QDG slice, and can remove the noisy gates a developer is trying to diagnose. A tool that offers either should label it as a view for one observable or one input and keep the QDG slice as the default.

What a slice cannot show: The QDG slice contained the fault in 95.7% of the units, and every unit it missed is an omission fault, a removed control, gate or preparation, or a gate moved to another qubit. No slice can contain an operation that the program lacks, nor follow an interaction that the fault has removed. When the slice of a qubit leaves out a qubit the developer expected it to depend on, that absence is itself the clue. A tool can make it visible by listing the qubits each slice contains and, by the duality of Proposition 5, the qubits each operation reaches, at no extra cost.

What remains open: Slices narrowed where to look but did not improve mutation-based fault ranking, and given a slice, no language model localized faults better than with the whole circuit by more than a fraction of a point. The models did miss dependences that a slicer finds by rule, and Sonnet recalled 0.61 and the larger Qwen model 0.16 of those through kickback, so a slice is better used to give a model or a developer the dependences than to rank suspects. Whether slices shorten the manual reading that developers report does not scale [4] is a question for a study with developers, which the semantics and the measured rules of this paper make possible to design.

VIII. Threats to Validity

[TODO: to be written]

IX. Conclusion

[TODO: to be written]

References

[1] M. Weiser, "Program slicing," *IEEE Transactions on Software Engineering*, vol. SE-10, no. 4, pp. 352–357, 1984.

[2] ——, "Programmers use slices when debugging," *Communications of the ACM*, vol. 25, no. 7, pp. 446–452, 1982.

[3] D. Binkley, "Source code analysis: A road map," in *Future of Software Engineering (FOSE '07)*. IEEE, 2007.

[4] J. Zappin, T. Stalnaker, O. Chaparro, and D. Poshyvanyk, "Challenges and practices in quantum software testing and debugging: Insights from practitioners," Preprint, 2026.

[5] M. Paltenghi and M. Pradel, "Analyzing quantum programs with LintQ: A static analysis framework for Qiskit," *Proceedings of the ACM on Software Engineering*, vol. 1, no. FSE, pp. 2144–2166, 2024.

[6] P. Zhao, X. Wu, Z. Li, and J. Zhao, "QChecker: Detecting bugs in quantum programs via static analysis," in *IEEE/ACM International Workshop on Quantum Software Engineering (Q-SE)*, 2023, pp. 50–57.

[7] H. Yu and J. Zhao, "The quantum program dependence graph and its uses in quantum software development," in *IEEE/ACM International Workshop on Quantum Software Engineering (Q-SE)*, 2025, pp. 48–55.

[8] M. Kaul, A. Küchler, and C. Banse, "A uniform representation of classical and quantum source code for static code analysis," in *IEEE International Conference on Quantum Computing and Engineering (QCE)*, 2023, pp. 1013–1019.

[9] J. M. Murillo, J. Garcia-Alonso, E. Moguel, J. Barzen, F. Leymann, S. Ali, T. Yue, P. Arcaini, R. Pérez-Castillo, I. García-Rodríguez de Guzmán, M. Piattini, A. Ruiz-Cortés, A. Brogi, J. Zhao, A. Miranskyy, and M. Wimmer, "Quantum software engineering: Roadmap and challenges ahead," *ACM Transactions on Software Engineering and Methodology*, vol. 34, no. 5, pp. 154:1–154:48, 2025.

[10] P. Murali, N. M. Linke, M. Martonosi, A. Javadi-Abhari, N. H. Nguyen, and C. H. Alderete, "Full-stack, real-system quantum computer studies: Architectural comparisons and design insights," in *International Symposium on Computer Architecture (ISCA)*, 2019, pp. 527–540.

[11] IBM Quantum, "LightCone transpiler pass, Qiskit SDK 2.4 documentation," https://eu-de.quantum.cloud.ibm.com/docs/en/api/qiskit/2.4/qiskit.transpiler.passes.LightCone, 2025, accessed 2026-10-29.

[12] P. Zhao, Z. Miao, S. Lan, and J. Zhao, "Bugs4Q: A benchmark of existing bugs to enable controlled testing and debugging studies for quantum programs," *Journal of Systems and Software*, vol. 205, p. 111805, 2023.

[13] K. J. Ottenstein and L. M. Ottenstein, "The program dependence graph in a software development environment," in *Proceedings of the ACM SIGSOFT/SIGPLAN Software Engineering Symposium on Practical Software Development Environments*, 1984, pp. 177–184.

[14] J. Ferrante, K. J. Ottenstein, and J. D. Warren, "The program dependence graph and its use in optimization," *ACM Transactions on Programming Languages and Systems*, vol. 9, no. 3, pp. 319–349, 1987.

[15] S. Horwitz, T. Reps, and D. Binkley, "Interprocedural slicing using dependence graphs," *ACM Transactions on Programming Languages and Systems*, vol. 12, no. 1, pp. 26–60, 1990.

[16] A. Podgurski and L. A. Clarke, "A formal model of program dependences and its implications for software testing, debugging, and maintenance," *IEEE Transactions on Software Engineering*, vol. 16, no. 9, pp. 965–979, 1990.

[17] B. Korel and J. Laski, "Dynamic program slicing," *Information Processing Letters*, vol. 29, no. 3, pp. 155–163, 1988.

[18] H. Agrawal and J. R. Horgan, "Dynamic program slicing," in *ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 1990, pp. 246–256.

[19] F. Tip, "A survey of program slicing techniques," *Journal of Programming Languages*, vol. 3, no. 3, pp. 121–189, 1995.

[20] H. W. Alomari, M. L. Collard, J. I. Maletic, N. Alhindawi, and O. Meqdadi, "srcSlice: very efficient and scalable forward static slicing," *Journal of Software: Evolution and Process*, vol. 26, no. 11, pp. 931–961, 2014.

[21] J. Silva, "A vocabulary of program slicing-based techniques," *ACM Computing Surveys (CSUR)*, vol. 44, no. 3, pp. 1–41, 2012.

[22] D. Binkley and M. Harman, "A survey of empirical results on program slicing," *Advances in Computing*, 2004.

[23] C.-K. Hur, A. V. Nori, S. K. Rajamani, and S. Samuel, "Slicing probabilistic programs," in *ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2014, pp. 133–144.

[24] T. Amtoft and A. Banerjee, "A theory of slicing for imperative probabilistic programs," *ACM Transactions on Programming Languages and Systems*, vol. 42, no. 2, pp. 1–71, 2020.

[25] F. Olmedo, "Slicing of probabilistic programs: A review of existing approaches," *ACM Computing Surveys*, vol. 58, no. 3, pp. 1–40, 2025.

[26] K. B. Gallagher and S. J. Kozaitis, "Program slicing: A brief retrospective," *IEEE Transactions on Software Engineering*, vol. 51, no. 3, pp. 720–724, 2025.

[27] J. A. C. Behler, A. F. Al-Ramadan, B. Baheri, Q. Guan, and J. I. Maletic, "Supporting static program analysis and transformation of quantum-based languages," in *IEEE International Conference on Quantum Computing and Engineering (QCE)*, 2024, pp. 836–842.

[28] J. A. C. Behler, A. F. Al-Ramadan, B. Baheri, M. L. Collard, Q. Guan, and J. I. Maletic, "Static analysis and transformation for quantum programming languages," *IEEE Software*, vol. 42, no. 5, pp. 58–64, 2025.

[29] N. C. L. Ramalho, H. A. de Souza, and M. L. Chaim, "Testing and debugging quantum programs: The road to 2030," *ACM Transactions on Software Engineering and Methodology*, vol. 34, no. 5, pp. 155:1–155:46, 2025.

[30] S. Perdrix, "Quantum patterns and types for entanglement and separability," *Electronic Notes in Theoretical Computer Science*, vol. 170, pp. 125–138, 2007.

[31] F. Prost and C. Zerrari, "Reasoning about entanglement and separability in quantum higher-order functions," in *Unconventional Computation (UC)*, ser. LNCS, vol. 5715, 2009, pp. 219–235.

[32] C. Yuan, C. McNally, and M. Carbin, "Twist: Sound reasoning for purity and entanglement in quantum programs," *Proceedings of the ACM on Programming Languages*, vol. 6, no. POPL, pp. 1–32, 2022.

[33] S. Perdrix, "Quantum entanglement analysis based on abstract interpretation," in *Static Analysis Symposium (SAS)*, ser. LNCS, vol. 5079, 2008, pp. 270–282.

[34] N. Yu and J. Palsberg, "Quantum abstract interpretation," in *ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2021, pp. 542–558.

[35] K. Honda, "Analysis of quantum entanglement in quantum programs using stabilizer formalism," *Electronic Proceedings in Theoretical Computer Science*, vol. 195, pp. 262–272, 2015.

[36] S. Xia and J. Zhao, "Static entanglement analysis of quantum programs," in *IEEE/ACM International Workshop on Quantum Software Engineering (Q-SE)*, 2023, pp. 42–49.

[37] A. JavadiAbhari, S. Patil, D. Kudrow, J. Heckey, A. Lvov, F. T. Chong, and M. Martonosi, "ScaffCC: Scalable compilation and analysis of quantum programs," *Parallel Computing*, vol. 45, pp. 2–17, 2015.

[38] M. Amy and V. Gheorghiu, "staq—A full-stack quantum processing toolkit," *Quantum Science and Technology*, vol. 5, no. 3, p. 034016, 2020.

[39] E. Farhi, J. Goldstone, and S. Gutmann, "A quantum approximate optimization algorithm," arXiv:1411.4028, 2014.

[40] L. Zhou, G. Barthe, J. Hsu, M. Ying, and N. Yu, "A quantum interpretation of bunched logic & quantum separation logic," in *Proceedings of the ACM/IEEE Symposium on Logic in Computer Science (LICS)*, 2021, pp. 1–14.

[41] X.-B. Le, S.-W. Lin, J. Sun, and D. Sanan, "A quantum interpretation of separating conjunction for local reasoning of quantum programs based on separation logic," *Proceedings of the ACM on Programming Languages*, vol. 6, no. POPL, pp. 1–27, 2022.

[42] B. Heim, M. Soeken, S. Marshall, C. Granade, M. Roetteler, A. Geller, M. Troyer, and K. Svore, "Quantum programming languages," *Nature Reviews Physics*, vol. 2, no. 12, pp. 709–722, 2020.

[43] K. N. Smith, G. S. Ravi, P. Murali, J. M. Baker, N. Earnest, A. Javadi-Abhari, and F. T. Chong, "TimeStitch: Exploiting slack to mitigate decoherence in quantum circuits," *ACM Transactions on Quantum Computing*, vol. 4, no. 1, 2022.

[44] S. A. Metwalli and R. Van Meter, "A tool for debugging quantum circuits," in *IEEE International Conference on Quantum Computing and Engineering (QCE)*, 2022, pp. 624–634.

[45] ——, "Testing and debugging quantum circuits," *IEEE Transactions on Quantum Engineering*, vol. 5, p. 2500415, 2024.

[46] IBM Quantum, "Qiskit addon utilities: `slicing` module," https://quantum.cloud.ibm.com/docs/en/api/qiskit-addon-utils/slicing, 2026, accessed 2026-10-26.

[47] N. H. Oldfield, C. Laaber, T. Yue, and S. Ali, "Faster and better quantum software testing through specification reduction and projective measurements," arXiv:2405.15450, 2024.

[48] Q. Chen, R. Câmara, J. Campos, A. Souto, and I. Ahmed, "The Smelly Eight: An empirical study on the prevalence of code smells in quantum computing," in *IEEE/ACM International Conference on Software Engineering (ICSE)*, 2023, pp. 358–370.

[49] P. K. Nayak, K. V. Kher, M. B. Chandra, M. V. P. Rao, and L. Zhang, "Q-PAC: Automated detection of quantum bug-fix patterns," arXiv:2311.17705, 2023.

[50] P. Zhao, J. Zhao, and L. Ma, "Identifying bug patterns in quantum programs," in *IEEE/ACM International Workshop on Quantum Software Engineering (Q-SE)*, 2021, pp. 16–21.

[51] E. Mendiluze, S. Ali, P. Arcaini, and T. Yue, "Muskit: A mutation analysis tool for quantum software testing," in *Proceedings of the 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2021, pp. 1266–1270.

[52] D. Fortunato, J. Campos, and R. Abreu, "QMutPy: A mutation testing tool for quantum algorithms and applications in Qiskit," in *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, 2022, pp. 797–800.

[53] A. García de la Barrera, I. García-Rodríguez de Guzmán, M. Polo, and J. A. Cruz-Lemus, "Quantum software testing: Current trends and emerging proposals," in *Quantum Software Engineering*, M. A. Serrano, R. Pérez-Castillo, and M. Piattini, Eds. Springer, 2022, pp. 167–191.
[54] Y. Ishimoto, M. Kondo, N. Ubayashi, Y. Kamei, R. Katsube, N. Sato, and H. Ogawa, "Evaluating mutation-based fault localization for quantum programs," in *Proceedings of the 29th International Conference on Evaluation and Assessment in Software Engineering (EASE)*, 2025, pp. 666–671.
[55] Qiskit Community, "The Qiskit textbook," https://github.com/qiskit-community/qiskit-textbook, 2024, accessed 2026-10-26.
[56] E. Bernstein and U. Vazirani, "Quantum complexity theory," *SIAM Journal on Computing*, vol. 26, no. 5, pp. 1411–1473, 1997.
[57] M. A. Nielsen and I. L. Chuang, *Quantum computation and quantum information*. Cambridge University Press, 2010.
[58] G. Canfora, A. Cimitile, and A. De Lucia, "Conditioned program slicing," *Information and Software Technology*, vol. 40, no. 11–12, pp. 595–607, 1998.
[59] N. Quetschlich, L. Burgholzer, and R. Wille, "MQT Bench: Benchmarking software and design automation tools for quantum computing," *Quantum*, vol. 7, p. 1062, 2023.
[60] A. Li, S. Stein, S. Krishnamoorthy, and J. Ang, "QASMBench: A low-level quantum benchmark suite for NISQ evaluation and simulation," *ACM Transactions on Quantum Computing*, vol. 4, no. 2, pp. 1–26, 2023.
[61] K. Chen, W. Fang, J. Guan, X. Hong, M. Huang, J. Liu, Q. Wang, and M. Ying, "VeriQBench: A benchmark for multiple types of quantum circuits," arXiv:2206.10880, 2022.
[62] J. Preskill, "Quantum computing in the NISQ era and beyond," *Quantum*, vol. 2, p. 79, 2018.